

Assignment: Build Something (Poll App: Software Description, Architecture Design, and Discussion)

Venkata Naga Sai Yaswanth Deevi
vede25@student.bth.se

1 Software Description

Poll App is a microservice-based application for polls and voting. A poll is a question with multiple options. The users can vote for one of the options in a poll, once per poll, per user. Once there are enough votes, any user can access live statistics, and the poll can be closed to prevent further voting.

The application has been purposely designed to be simple in its domain logic since the purpose of this assignment is to teach how to design microservices, which communicate with each other, as well as how to deploy them with Kubernetes. The key thing here is that the application has to be decomposed into independent microservices which communicate only via REST and can be scaled independently from each other.

The application consists of four microservices, all written in Node.js with the Express framework, and one MongoDB database as in the course material:

- **poll-service:** creates, lists, retrieves, and closes polls.
- **vote-service:** records votes against a poll, enforcing one vote per voter per poll.
- **results-service:** computes a live tally of votes for a given poll. Holds no state of its own.
- **notification-service:** records a notification whenever a poll is closed.

REST APIs are used by all the four services to communicate between themselves as well as to the outside world using just plain HTTP. Code sharing does not exist between any of the four services. Every service is a separate Docker container that can be started, stopped, and scaled independently of the others.

2 Software Architecture Design

2.1 Component Overview

Component	Port	Responsibility
poll-service	3000	Owns the <code>polls</code> collection. Create, list, read, and close polls.
vote-service	3001	Owns the <code>votes</code> collection. Validate and record votes.
results-service	3002	Stateless read model. Aggregates data from poll-service and vote-service.
notification-service	3003	In memory log of poll closed events. No persistent storage.
mongo	27017	Shared database, backing poll-service and vote-service.

Table 1: Microservices and their responsibilities

2.2 Interaction Diagram

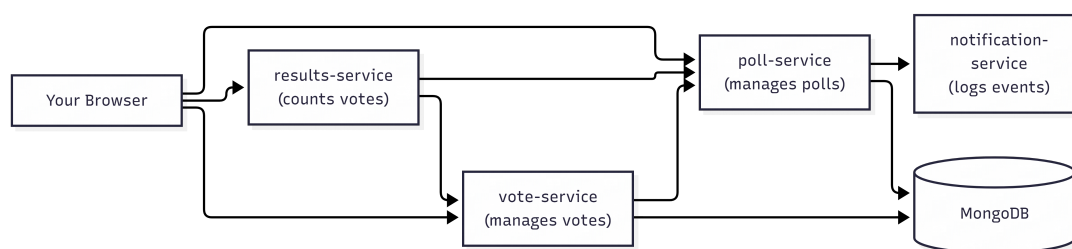


Figure 1: Interaction between the four microservices and the database

Each line in the above diagram is simply an HTTP REST API call. Neither shared memory nor message queues are used; also, there's no access to the database directly from service to service. Poll-service is the only service that both reads and writes the polls database, while vote-service is the only service that both reads and writes the votes database. The results-service doesn't hold any of its own data; rather, it generates data for each of its requests by invoking poll-service and vote-service.

2.3 Component to Microservice Mapping

Conceptual Component	Implementing Microservice
Poll management (create, list, read, close)	poll-service
Vote casting and duplicate vote prevention	vote-service
Live result computation	results-service
Poll closed notifications	notification-service
Persistent poll and vote storage	mongo (MongoDB)

Table 2: Mapping from application concepts to deployed microservices

2.4 Cloud Architecture Patterns Applied

Horizontally Scaling Compute Pattern

results-service is completely stateless and read-only. It does not store any data across requests, and therefore an unlimited number of replicas could be serving traffic simultaneously without any synchronization. That is the part of the application that is designed for scaling-out most.

Colocate Pattern

All applications and the database work on the same Kubernetes cluster and communicate within the cluster's network, thus maintaining a low latency for collaborating applications.

Separation of Concerns between provisioning and orchestration

Dockerfiles for each individual service exist; however, their job is limited to the creation of the execution environment only. The orchestration (replication, networking, storage), on the other hand, is managed entirely through Kubernetes manifests, and not through Dockerfiles and the application code itself.

Avoidance of the Queue Centric Workflow Pattern

Purposefully, the app does not have a message queue between the two services. A poll closing notification is pushed directly by using a synchronous REST API call to the notification service without going through the message queue. There was a purpose for that: It is a very simple, one-sided operation, which doesn't require any kind of persistence through possible restarts of the client; therefore, the use of a message queue adds nothing but operational overhead.

3 Discussion: Benefits, Challenges, and Security

3.1 Benefits of this Architecture

- **Independent scaling.** As there is no service which maintains a state associated with a particular request session (there is no socket.io and there are no sticky sessions), any particular service can be independently scaled up and down. This is evident from the fact that the results-service has been scaled from three to five replicas without any code changes or downtime.
- **Independent deployability.** Each individual service has its own unique Dockerfile, and each has its own image on Docker hub. The services can be built, deployed, and pushed independently without affecting other services like poll-service, vote-service, and notification-service.
- **Clear ownership of data.** One service needs to communicate to a specific MongoDB collection at any point of time. This will prevent the common microservices antipattern of having multiple services sharing the same database schema and therefore getting coupled through it despite their individual deployments.

3.2 Challenges

- **Increased operational surface.** Four services and one database translate into four different manifest files, logs and potential failure points. Locating a particular error during development involved analyzing logs of three separate services.
- **Distributed failures are harder to diagnose.** The biggest time waster was an unhandled promise rejection in poll-service which crashed the entire process when MongoDB became inaccessible temporarily. It looked like a problem with Kubernetes networking at first, but only pod crash logs could provide a clue.
- **No transactional guarantees across services.** In case vote-service makes a record of a vote, but a subsequent operation fails, there are no rollback mechanisms provided. This is the tradeoff of the microservice approach which is acknowledged.
- **This application is too small to genuinely need scaling.** One instance of each of these services can handle much larger amounts of traffic than it will generate. Scaling of results-service only makes sense if there is a lot of simultaneous polls or a much heavier computation needed, say counting millions of votes.

3.3 Security Discussion

Several security shortcuts were taken deliberately, since hardening was not the focus of the assignment.

No authentication on any service

Polls and voting can be done by anyone accessing the poll-service or vote-service using any voterId, as voterId is an unchecked string that caller passes. In a production environment, one should implement the authentication via signed tokens issued at login by vote-service.

No rate limiting

One client can create unlimited polls or votes. Rate limiting should be implemented either in each service or by API Gateway.

MongoDB has no authentication enabled

MongoDB works without any credentials and can be accessed by any pod from the cluster.

In a production environment, one would have to use keyfile for internal authentication and application user with credentials stored as Kubernetes Secret, as described in course materials.

Services trust each other's responses without verification

Vote-service checks poll-service about the state of a poll and its options, results-service trusts both services. While it seems to be reasonable in a closed network of a cluster, a malicious service could provide fake data to other services.

MongoDB bound to all interfaces

By default mongod binds to 127.0.0.1, but it was changed to 0.0.0.0 during the development process. Without any authentication, it is okay only due to the closed nature of the cluster's internal network. Making MongoDB service type to be LoadBalancer would make it vulnerable.

None of these were exploited during this project. They are documented as known, accepted gaps for an exercise of this scope.

4 Repository

The source code of all the microservices, along with the docker-compose.yml file and the kubernetes manifests, are located in the git repository of the project.

<insertrepositoryURLhere>

Disclosure

All content in this report and all source code for the application were written by me. AI tools were used only to format this documentation for a professional presentation and to assist with debugging when I was hard stuck.